Agario Game Clone Report

Musa Jawad
Arne Hilao
Usharab Khan
Savitur Maharaj
Rian Tapang

Introduction

This project aims to replicate, at least in part, the functionality of the game Agar.io. Agar.io is a multiplayer game in which multiple players(blobs) join a lobby and try to grow by eating food (dots). Our goal was to create a distributed system which could handle the real-time multiplayer of multiple players using a single server architecture. Multiplayer games require delicate attention to the synchronization, consistent game state, and low latency. As such we attempted to create a basic version of Agar.io to explore these concepts of distributed systems.

Figure - 1: An agar.io game with arrows showing food and players

Architecture

Our project uses Phaser and Typescript for the frontend. Phaser is a HTML5 game framework which allows easier game development for web projects. We decided to use Phaser as creating a game engine and all its components are both out of the scope of this project and

highly unnecessary. By using Phaser we can create the frontend features for our Agar.io game much easier than if we did using just react. Using Phaser as our skeleton, we also used typescript to create our frontend client in order to send information to our proxy server.

For the backend we decided to use Django. We decided to use django as it is a high level framework which allowed us to create multiple replicas in the same project. This help simplify the connection process. We used Django for all our replicas and their functionality. This includes connections, database access, calculating new game states, and other smaller features. For our database we decided to use SQL. SQL allowed us to store our player information in a persistent location and allowed easy access with its Django integration.

Our overall architecture is split into 3 layers. Our first layer is the client. The client is the frontend for the user and is tasked with sending player information to the proxy as well as updating its users screen with the updated information received from the proxy.

Our second layer is the proxy server. The proxy server acts as a relay between the client and the primary replica. It also handles items such as leader election and primary replica connections. The proxy sends the data received from the client, and also sends data back to the client of the updated player information and food list.

Our final layer is the backend layer. The backend layer contains the primary replica, the backup replicas, and all their respective databases. The primary replica receives the data from the proxy server and does calculations for new player locations and food lists. It does this based on previous player locations (which are retrieved from the database) and their current velocity. After it calculates this new information it stores this data into its database. Once it updates its own database, it propagates these changes to the backup replicas. Each of these backup replicas update their respective databases with this new information.

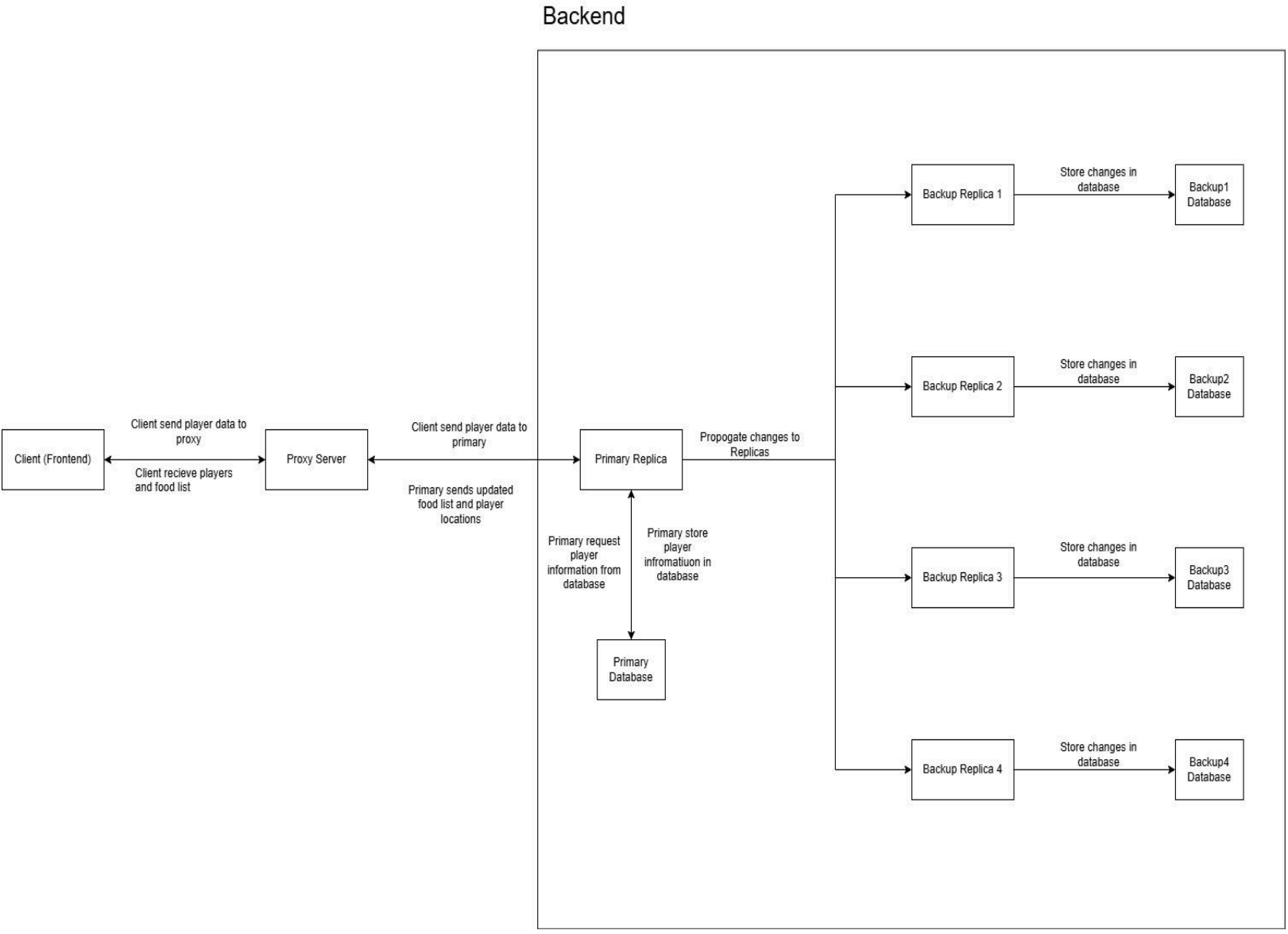

Figure 2: Diagram showing system architecture and connections.

Communication

Our project uses two methods to communicate between processes, HTTP requests to handle async requests like health checks, and leader election, as well as websocket connections to handle data that needs more synchronicity, such as player input and propagation of game-data and logic.

Synchronization

Our system prefers to have a slightly more transparent system than more distributed systems. Because it is a game, we decided that proper player feedback was more important than maintaining the illusion of a vast distributed network. As such, we have implemented roll-backs.

Our primary replica updates the backup replicas every half second, and whenever the primary replica crashes, the player is informed that the primary server has crashed, and pauses all input, while placing a message to inform them of it. It does this by using lamport timestamps. These timestamps will be further explored in consistency.

```
133     # Game tick loop:
134     async def game_loop(self):
135         global EVENT_QUEUE
136 
137         try:
138             while True:
139                 await asyncio.sleep(1/ TICK_RATE)
140 
141                 # Process events in the order of timestamps:
142                 if len(EVENT_QUEUE) > 0:
143                     event = EVENT_QUEUE.dequeue()
144                     await self.process_event(event)
145                     # Send event to backup replicas:
146                     await propagate_event_to_replicas(event)
147                 else:
148                     pass
149 
150                 curr_player = await get_player_position(self.player_id)
151 
152                 await self.broadcast({
153                     "type": "update",
154                     "id": self.player_id,
155                     "x": curr_player["x"],
156                     "y": curr_player["y"]
157                 })
158 
159         except asyncio.CancelledError:
160             pass
161 
```

Figure 3: Code depicting the EVENT_QUEUE, which is a queue which holds the data being saved to the primary. Once it is dequeued and saved, it calls another function called propogate_event_to_replicas() to send data to each replica. This is done every 1/TICK_RATE. Current it is set such that it makes updates every half second

This is to ensure that the player experience is as enjoyable as possible. At first, we thought about a full-rollback, where we would allow players to continue moving client side, and then rollback to the server-tracked position when the primary replica gets updated.
We felt that, not only was it practically the same as simply pausing the game, it would be far more frustrating to lose what might have potentially been a big leap in score, than stop it from happening at all. That is, the perceived loss of a win would be more frustrating than the momentary frustration of the game pausing.

We also thought about perhaps some level of interpolation to reconcile the server's position with the client's position, but we ultimately decided that it would be similarly frustrating in the edge-case that a primary replica takes longer than a couple seconds to be elected.

Replication

Our project uses passive replication, with our servers having one primary replica, and multiple backup replicas that periodically get updated, just in case of a crash. These replicas have one primary replica which handles all of the game-logic, and which handles the propagation of information to the backups via websockets forwarding received information.

The primary can then keep track of the replicas which maintain a connection, pruning it if it detects a crash from the websockets. As well as that, on the leader's crash, it sends out a heart-beat

```python
# Check if this server is the primary replica
def is_primary():
    global THIS_SERVER
    current_primary_server = get_primary()
    return THIS_SERVER == current_primary_server


# Send the POST request to the other replicas if you are the primary
def propagate_to_replicas(data):
    global SERVERS
    global THIS_SERVER
    for server in SERVERS:
        if server != THIS_SERVER:
            try:
                requests.post(f'http://{server}/replica/create_player/', data=data, timeout=2)
            except requests.exceptions.RequestException:
                print(f"Server did not respond: {server}")
```

Figure 4: Code for checking if the current server is the primary. Function is_primary() checks global list and checks if it is primary. Based on if it is primary or not, this server's responsibilities and code run will be different.

Election

Our project uses an election algorithm in which the proxy can detect the primary replica crashing, and from there it sends health checks to every other replica. Replicas that reply before the timeout are maintained as "healthy", and those that don't are removed from the list of tracked servers.

Then it'll find the replica with the highest id and turn that into the leader.

As such, it doesn't really have a proper process-to-process election, as we didn't see the need for one while we maintained a single point of failure in the form of the proxy. We had plans to introduce multiple proxies which could be load-balanced and handle election among themselves, but we were primarily worried about making sure that the game-state could survive

a replica crashing, and thus focused on that. We understand this is a particularly large flaw in our current design.

Consistency

Because our project is a game, we decided that we would only have one process at any given time handling the calculations, which is where we had the idea for the proxy and passive-replication methods we currently employ because we knew we needed high-consistency. Our replicas all needed to save the game-state, and agree with one another, at least on the order that read and write operations happen.

It was for this reason we chose a **Sequential Consistency** model. We knew that the replicas all needed to ensure a total order, even if it meant discarding events that happened out of order, as player-experience would be soured if, for example, the primary replica showed them eating a player, and a backup replica showed that they were the ones eaten instead. If the primary crashes in that scenario, and that backup was chosen as the new primary, the player would go from “winning” to “lost”, which we decided was too frustrating an experience for the user.

As such, we use timestamps to force the replicas to agree with one another. Each request made by the frontend contains a timestamp that the proxy processes and then passes along to the primary replica.

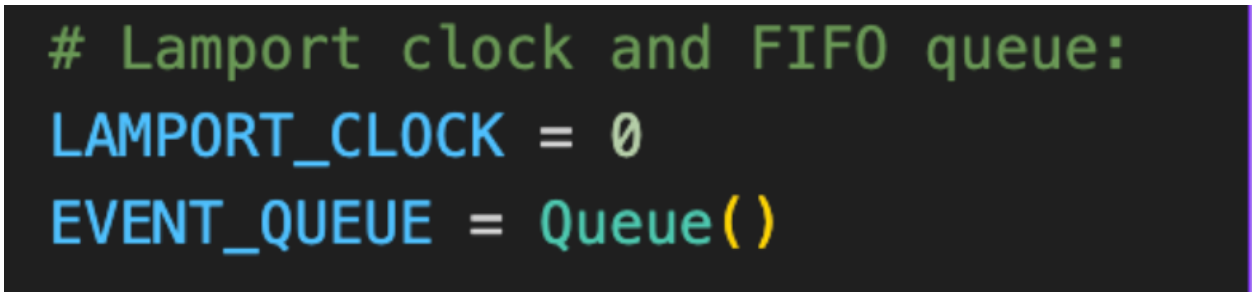

Figure 5: Code showing initialization of Lamport timestamps and event queue.

The primary replica ensures that the process isn’t received out of order by discarding any message with a timestamp less than or equal to the currently tracked timestamp, and then propagates that information to the backups. From there, the backups will receive it, and do the same thing. If they receive a message with an out of order timestamp, it will discard the information.

```python
async def receive(self, text_data):
    global EVENT_QUEUE
    global LAMPORT_CLOCK
    # If the player does not exist return:
    try:
        await get_player(self.player_id)
    except:
        return

    event = json.loads(text_data)
    timestamp = event.get("timestamp")
    print(timestamp)
    # Ignore events without a timestamp (ping):
    if timestamp is None:
        return

    # Update local lamport clock
    # If event's timestamp is newer, add it to the queue:
    if timestamp > LAMPORT_CLOCK:
        LAMPORT_CLOCK = timestamp
        EVENT_QUEUE.enqueue(event)
    else:
        print(f"Discarded event with out-of-order timestamp: {timestamp}")
```

Figure 6: Example case of using lamport and EVENT_QUEUE. An event will only be queued in the EVENT_QUEUE if the timestamp is greater than the current local timestamp.

By doing this, we ensure that no outdated requests are recorded by the replicas. This means that the most recent message the backup replica sees is *always* the most up-to-date it has received so far. Which means no data is accidentally overwritten by an outdated request, and thus no data that is wrong gets displayed to the user.

Fault Tolerance

As mentioned above, our system uses roll-backs and redundancy for its fault-tolerance. We understood that, as a game, our players expect 100% reliability. To that end, we have the game-state saved over multiple machines, each one ready to take over if necessary.

We know that the primary replica crashing would cause a perceived “lag spike”, wherein the player is jarringly rubber-banded back to the server-tracked position, but we decided this was acceptable as opposed to the alternative, where the entire game-state is lost entirely on primary replica failure. In this way, a momentary setback would leave the user mildly frustrated, as opposed to considerably frustrated at losing the game entirely.

Unfortunately, we *do* have a single-point of failure in the proxy, but we believed this was acceptable because this was how game servers run by larger companies operate: one primary point of contact for a given region.

Given time, we would have implemented a load-balancer solution, where we would implement multiple proxies and have them seamlessly check on each other, and take over the primary proxy role if the proxy started taking too long to respond, but given our focus on ensuring the health of the replicas, we hadn't been able to look at pursuing this solution.

Summary and Conclusions

Our system still maintains a flaw in the single-point of failure that the proxy provides. Because we hadn't implemented a system with multiple proxies, we were only able to have one proxy, which we treated sort of like a name registry to keep track of all other replicas.

But because we only have the one server, we can keep a static reference to the proxy so that crashed replicas can eventually reconnect with the proxy, and rejoin as a backup. This also means that we can increase the number of replicas to some arbitrary number, however the single point of failure is a fatal flaw of our system currently, as there is no way for the system to survive the crash of the proxy.

We also have a flaw in the speed of the propagation. Because we continuously update our replicas, it could cause the system to slow down with an increased amount of players. We could mitigate this by increasing the multi-threading of our primary server, or perhaps offloading the burden to the replica servers to maintain a section of the data, but this would require a large overhaul of our current architecture that we didn't have time for.